

```

#####
#exercice 1#
#####
"""le schéma relationnel
train(immatriculation,gare_attache,#id)
-----
trajet(num_trajet,#immatriculation,ville_dep,ville_arr,heure_d
-----
type(id, nom,nb_place)
--
"""

#Q1
"""
SELECT heure_dep, heure_arr, immatriculation
FROM TRAJET
WHERE ville_dep='Fez';
"""

#Q2
"""
SELECT DISTINCT TR.heure_dep,TR.heure_arr,TR.immatriculation,
FROM TRAJET TR JOIN TRAIN TRA ON TR.immatriculation=TRA.immat
WHERE TR.ville_dep='Fez';
"""

#Q3
"""
SELECT TRA.id, COUNT(TR.num_trajet)
FROM TRAJET TR JOIN TRAIN TRA ON TR.immatriculation=TRA.immat
WHERE ville_dep='RABAT'
GROUP BY TRA.id;
"""

#Q4
"""
SELECT TR.ville_dep, TY.nom
FROM TRAJET TR JOIN TRAIN TRA ON TR.immatriculation=TRA.immat
WHERE TR.heure_dep>='07h00' and heure_arr<='13h00';
"""

#Q5
"""
import sqlite3
db=sqlite3.connect("C:\\CNC\\gestion_trains.sqlite")
c=db.cursor()
c.execute("SELECT heure_dep,heure_arr,immatriculation FROM TR
for tup in c:
    print(tup[0],tup[1],tup[2])
"""

```

```

db.close()
"""
#####
#exercice 2#
#####
#Q6
"""posons  $Y(t)=(y(t),y'(t))$ 
donc  $Y'(t)=(y'(t),y''(t))=F(t,Y(t))$ 
par conséquent  $Y'(t)=(y'(t),-3*y(t)-y'(t))=F(t,Y(t))$ 
avec  $Y(a)=(y(a),y'(a))$  (1)"""
#Q7
import numpy as np
def F(z,t):
    """ $z=(y(t),y'(t))$  et la variable  $t$  est le paramètre  $t$  de
    return(np.array([z[1],-3*z[0]-z[1]]))
#Q8
import matplotlib.pyplot as pp
import numpy as np
import scipy.integrate as si
t=np.linspace(0,5,100)
Y=si.odeint(F,[0.05,0.7],t) #[0.05,0.6]ou [0.05,0.5]
Y1=[]
for l in Y:
    Y1+=[l[0]]
pp.axis([0,5,-0.16,0.3])
pp.xlabel("t")
pp.ylabel("y")
pp.plot(t,Y1,'b')
#pp.savefig("cnc2015")
#pp.show()
#####
#Problème #
#####
#Q9
""" Avec la notation post-fixe la priorité entre les diffère
#####
#première par
#####
#####
#Pour l'implémentation de votre Pile vous devez utiliser le
#####
def Len(t):
    c=0

```

```

        for e in t:
            c+=1
        return(c)
#Q10
def initPile():
    return([])
#Q11
def estVide(pile):
    if Len(pile)==0:
        return(1)
    return(0)
#Q12
def empiler(pile,elem):
    pile.insert(0,elem)
#Q13
def depiler(pile):
    if estVide(pile):
        raise PileVideException ("pile vide")
    else:
        p=pile[0]
        pile.remove(pile[0])
        return(p)
#Q14
def valeurSommet(pile):
    if estVide(pile):
        raise PileVideException ("pile vide")
    else:
        return(pile[0])
#Q15
def hauteur(pile):
    return(Len(pile))

```

```

#####
#deuxième par
#####

```

```

#Q16
def estEntier(x):
    def estEntier(x,n):
        if n==1:
            if x[n-1]>='1' and x[n-1]<='9': return(True)
            return(False)
        else:
            etat=estEntier(x,n-1)
            return(etat and (x[n-1]>='1' and x[n-1]<='9'))
    return(estEntier(x,Len(x)))

```

```

        return(estEntier(str(x),Len(str(x))))
#test
#print(estEntier(12),estEntier('12'),estEntier('1a2'))
#Q17
def eval(op,e1,e2):
    if op not in {'+', '-', '*', '/', '.'}:
        raise OpNonValideException ("op@rateur invalide")
    elif type(e1)!=int or type(e2)!=int:
        raise ArgNonValideException("argument invalide")
    elif op=='+': return(int(e1)+int(e2))
    elif op=='-': return(int(e1)-int(e2))
    elif op=='*': return(int(e1)*int(e2))
    elif op=='.': return(str(e1)+op+str(e2))
    else:
        if int(e2)!=0: return(int(e1)/int(e2))
        else:
            raise ZeroDivisionError("division par zero")
#test
#print(eval('.',1,3))
#Q18
def evaluate(expr):
    pile=initPile()
    i=0
    while expr[i]!='.':
        if expr[i] in {'+', '-', '*', '/'}:
            e1=depiler(pile)
            e2=depiler(pile)
            e3=eval(expr[i],e1,e2)
            empiler(pile,e3)
        else:
            empiler(pile,expr[i])
        i+=1
    return(depiler(pile))
#test
#print(evaluate([7,4, '+', '.']))
#Q19
def estBienParenthesee(expr):
    pile=initPile()
    i=0
    while (expr[i]!='.'):
        if expr[i]=='(':
            empiler(pile,')')
        elif expr[i]==')':
            if estVide(pile)==0 and valeurSommet(pile)==expr[

```

```

        depiler(pile)
    else:
        return("Expression mal parenthes]@e")
    i+=1
    if estVide(pile)==0:
        return("Expression mal parenthes]@e")
    else:
        return("Expression bien parenth]@]s@e")
#test
#print(estBienParenthesee(['(', '1', '+', '(', '5', '*', '(', '6', '+
#Q20
def transInfix(expr):
    if Len(expr)==0:
        print("expression vide")
    else:
        erreur=False
        i=0
        pile1=initPile()
        pile2=initPile()
        while expr[i]!='.' and not erreur:
            if expr[i] in {'+', '-', '*', '/'}:
                empiler(pile1,expr[i])
            elif estEntier(expr[i]):
                empiler(pile2,expr[i])
            else: erreur=True
            i+=1
        if erreur:
            print("expression mal form]@e")
        else:
            L=[]
            L.append(depiler(pile2))
            while not estVide(pile1) and not estVide(pile2):
                L.append(depiler(pile2))
                L.append(depiler(pile1))
            L.append('.')
            return(L)
#test
#print(transInfix(['1', '+', '3', '*', '4', '+', '8', '. ']))
#Q21
"""

```

id]@e:

on propose d'utiliser deux files: une file pour les op]@randes
explication:

on parcourera la liste, on empilera les opérandes dans une fi

```
def transInfix1seulNniveau(L):
    pile1=initPile()
    pile2=initPile()
    i=0
    while(L[i]!='.'):
        if estEntier(L[i]):
            empiler(pile1,L[i])
        elif L[i] in {'+', '-', '*', '/'}:
            if estVide(pile2)==0:
                if valeurSommet(pile2) in {'*', '/'}:
                    op=depiler(pile2)
                    empiler(pile2,L[i])
                    empiler(pile2,op)
                else:empiler(pile2,L[i])
            else:empiler(pile2,L[i])
        i+=1
    L1=[]
    while hauteur(pile1)>1 and hauteur(pile2)>0:
        L1.append(pile1.pop(-1))
        L1.append(pile1.pop(-1))
        L1.append(pile2.pop(-1))
    if hauteur(pile1)!=0:
        L1.append(pile1.pop(-1))
    if hauteur(pile2):
        L1.append(pile2.pop(-1))
    L1.append('.')
    return(L1)

#test
#print(transInfix1seulNniveau(['(', '1', '+', '2', ')', '*', '(', '4
#Q22
def transtotal(L):
    pile1=initPile()
    pile2=initPile()
    L1=[]
    i=0
    while L[i]!='.':
        if estEntier(L[i]):
            empiler(pile2,L[i])
        else:
            empiler(pile1,L[i])
        i+=1
    while not estVide(pile2):
        L1.append(depiler(pile2))
```

```

    pile1.reverse()
    while not estVide(pile1):
        L1.append(depiler(pile1))
    L1.append('.')
    return(L1)
#print(transtotal(['8', '4', '+', '3', '*', '1', '-', '.']))
#Q23
def transtotal1(L2):
    L=transInfix(L2)
    pile1=initPile()
    pile2=initPile()
    L1=[]
    i=0
    while L[i]!='.':
        if estEntier(L[i]):
            empiler(pile2,L[i])
        else:
            empiler(pile1,L[i])
        i+=1
    while not estVide(pile2):
        L1.append(depiler(pile2))
    pile1.reverse()
    while not estVide(pile1):
        L1.append(depiler(pile1))
    L1.append('.')
    return(L1)
#print(transtotal(['1', '+', '3', '*', '4', '+', '8', '.']))

```